designlang — How to Use It

> **Package:** `designlang` (npm) · **Repo:** Manavarya09/design-extract · **Version installed:** `9.0.0` · **Guide written:** 22/04/2026

A step-by-step guide for a brand-new developer. If you can open Terminal and paste commands, you can use this tool. Every command in this guide is copy-paste ready.

1. What designlang actually does

You give it a website URL. It opens that site in a headless (invisible) Chrome browser, reads every computed style the browser applied, and spits out a complete **design system** you can drop into your own project:

- Colours, fonts, spacing, shadows, border radius, breakpoints
- **DTCG design tokens** (the W3C standard format — works with Figma, Style Dictionary, everything)
- A **Tailwind v4** config file
- A **shadcn/ui** theme
- CSS custom properties (variables)
- Figma Variables JSON (paste into Figma)
- React/TypeScript component stubs with the right types
- Motion tokens (easing, duration)
- A **brand voice** fingerprint (tone, CTA verbs)
- An **accessibility audit** (WCAG contrast failures listed)
- Agent rule files for Claude Code, Cursor, Windsurf — so the AI knows the brand without you explaining

Think of it as: *"I like how stripe.com looks. Give me a starter kit that matches."*

2. What's already installed on this machine

As of 22/04/2026, the following are done. You don't need to do these again.

```
designlang 9.0.0          → /opt/homebrew/bin/designlang
Playwright Chromium       → ~/Library/Caches/ms-playwright/
Claude Code skill         → ~/.claude/skills/extract-design/
```

Test it by running:

```
designlang --version
```

You should see `9.0.0`. If you get `command not found`, skip to §9 Troubleshooting.

---

3. Your first extraction (the 30-second version)

Open Terminal. Type this. Press Enter.

```
cd ~/Desktop
designlang https://stripe.com
```

In 30–60 seconds you'll see a new folder called `design-extract-output/` with ~20 files inside. Open `stripe-com-preview.html` in your browser first — it's a visual swatch report of every colour, font, and component it found.

That's it. You now have Stripe's design system as machine-readable files.

---

4. Reading the output files (what each one is for)

Files are named `<domain>-<type>.<ext>`. For `stripe.com` you get:

FILE	WHAT IT IS	WHEN TO USE IT
`*-preview.html`	Visual swatch report in your browser	**Open this first.** Eyeball what was found.
`*-design-language.md`	Everything, written for AI to read	Paste into Claude/Cursor as context.
`*-design-tokens.json`	DTCG W3C standard tokens	Import into Figma, Style Dictionary, any tool.
`*-tailwind.config.js`	Tailwind v4 theme	Drop into a Next.js/Tailwind project.
`*-variables.css`	CSS custom properties	Paste into `globals.css`.
`*-shadcn-theme.css`	shadcn/ui theme vars	For a shadcn/ui project.
`*-figma-variables.json`	Figma Variables import	Figma → Variables → Import.
`*-theme.js`	React/CSS-in-JS theme object	`styled-components`, `emotion`, etc.
`*-anatomy.tsx`	Typed React component stubs	Starter `<Button>`, `<Card>`, etc.
`*-motion-tokens.json`	Easing + duration tokens	For animations.
`*-voice.json`	Brand tone & CTA verbs	Copywriting reference.
`*-mcp.json`	MCP companion metadata	For Claude Code / Cursor.
`screenshots/`	Component screenshots	Visual reference.

5. The one flag that matters most: `--emit-agent-rules`

If you're using Claude Code (you are — you're reading this guide Claude made), **always pass this flag.** It tells designlang to also write:

- `.claude/` folder with Claude-specific rules
- `.cursor/rules/` for Cursor
- `agents.md` — a portable agent briefing

```
designlang https://stripe.com --emit-agent-rules
```

Now when you start a Claude Code session in that folder, Claude automatically knows the brand's colours, fonts, tone, and rules. You don't have to explain "use this blue, not that blue" — it reads the rule file.

6. Common command recipes

Copy-paste any of these. Replace the URL with whatever you want.

A) Full pipeline — screenshots + responsive + hover states + dark mode

```
designlang https://vercel.com --full --dark --emit-agent-rules
```

B) Crawl 5 pages deep (gets more components, averages tokens)

```
designlang https://stripe.com --depth 5 --emit-agent-rules
```

C) Multi-platform — also generate iOS Swift, Android, Flutter, WordPress theme

```
designlang https://stripe.com --platforms ios,android,flutter,wordpress
```

D) A page behind a login (export cookies from your browser first)

```
designlang https://internal.company.com --cookie-file ./session.json
```

To get `session.json`: in Chrome, install a cookies.txt exporter extension, export the site's cookies, save the file.

E) Lint an existing token file (checks for colour sprawl, contrast fails)

```
designlang lint ./src/tokens/design-tokens.json
```

Exits non-zero on errors. Good for CI.

F) Compare your local tokens vs the live site (drift detection)

```
designlang drift https://yourapp.com --tokens ./src/tokens.json --tolerance 8
```

Tells you: `in-sync` / `minor-drift` / `notable-drift` / `major-drift`.

G) Visual diff two URLs side-by-side

```
designlang visual-diff https://staging.yourapp.com https://yourapp.com
```

Produces a single-file HTML report with screenshots + colour deltas.

H) Compare multiple brands at once

```
designlang brands stripe.com vercel.com linear.app
```

I) Watch a site for changes (polls every 60s)

```
designlang watch https://stripe.com --interval 60
```

---

7. Using it through Claude Code (the skill)

The skill `extract-design` is already registered. Just say one of these in Claude Code:

- `/extract-design https://stripe.com`
- `extract design from https://vercel.com`
- `what colours does linear.app use`
- `get the design system from notion.so`

Claude runs `designlang` for you and reads the outputs. You don't touch the terminal.

---

8. Running it as an MCP server (power-user mode)

An MCP server means Claude Code (or Cursor / Windsurf) can query designlang *without re-running the whole extraction*. It exposes 5 resources and 5 tools.

Start the server

```
# First, extract a site so there's data to serve
cd ~/Desktop/designlang-guide
designlang https://stripe.com

# Now start the MCP server pointed at those outputs
designlang mcp --output-dir ./design-extract-output
```

Leave that terminal running. The server speaks JSON-RPC over stdio.

Connect it to Claude Code

Edit `~/.claude.json` (or use the `/mcp` command in Claude Code) and add:

```
{
  "mcpServers": {
    "designlang": {
      "command": "designlang",
      "args": ["mcp", "--output-dir", "/Users/YOU/Desktop/designlang-guide/design-extract-out
    }
  }
}
```

Restart Claude Code. You'll now have these **resources** Claude can read:

- `designlang://tokens/primitive` — raw colour/font/space values
- `designlang://tokens/semantic` — `bg-primary`, `text-muted`, etc.
- `designlang://regions` — header, footer, nav regions
- `designlang://components` — detected components (button, card, etc.)
- `designlang://health` — WCAG + design health score

And these **tools** Claude can call:

- `search_tokens` — find any token by name/value

- `find_nearest_color` — "what's the closest brand colour to `#4F46E5`?"
- `get_region` — fetch one region
- `list_failing_contrast_pairs` — WCAG fails

---

9. Chrome extension (one-click extraction from any tab)

The repo ships a Chrome extension. Chrome security rules mean **you have to click "Load unpacked" yourself** — this is the one step nobody can automate for you.

Step-by-step:

1. Clone the repo somewhere:

```
git clone https://github.com/Manavarya09/design-extract.git ~/Desktop/designlang-src
```

2. Open Chrome. In the address bar type: `chrome://extensions` → Enter.
3. Top-right, toggle **Developer Mode** ON.
4. Click **Load unpacked**.
5. Navigate to `~/Desktop/designlang-src/chrome-extension/` → click **Select**.
6. A new designlang icon appears in your toolbar. Click it on any site to trigger extraction.

---

10. Troubleshooting

`designlang: command not found`

The npm global bin isn't symlinked into `/opt/homebrew/bin`. Fix:

```
ln -sf $(npm prefix -g)/bin/designlang /opt/homebrew/bin/designlang
```

`Error: browserType.launch: Executable doesn't exist at ...chromium...`

Playwright browsers missing. Fix:

```
cd $(npm root -g)/designlang && node_modules/.bin/playwright install chromium
```

`Extraction returns only 1 colour / blank output`

The site is rendered entirely client-side and designlang gave up too fast. Retry with:

```
designlang https://the-site.com --full --wait 8000
```

`--wait` in ms tells it to wait after load before scraping.

`HTTPS / SSL error` on a staging/internal site

Add `--insecure` to skip cert validation:

```
designlang https://staging.local --insecure
```

`Permission denied` writing output

You're in a folder you don't own. `cd ~/Desktop` first.

The output folder name conflicts with a previous run

designlang writes into `./design-extract-output/` by default. It appends, doesn't wipe. To force a clean run:

```
rm -rf ./design-extract-output
designlang https://example.com
```

Or pass a custom dir:

```
designlang https://example.com --output-dir ./stripe-extraction
```

11. When NOT to use it

- **Private/authenticated pages with MFA** — designlang can't do MFA. Use it against public pages or pages with session-cookie-only auth.
- **Single-page apps that need user interaction to render content** — e.g. a dashboard that only shows data after you click. Use `--auto-interact` or pass `--full`, but expect gaps.
- **Sites behind Cloudflare Turnstile / hCaptcha** — the headless browser gets blocked. No good workaround.
- **iGreat-branded surfaces** — you already have `~/.claude/skills/igreat-design-system/` which is the authoritative source. Don't overwrite it with a scrape.

---

12. Cheat sheet (tape this above your desk)

```
# Quick
designlang <url>

# Quality
designlang <url> --full --emit-agent-rules

# Multi-platform
designlang <url> --platforms ios,android,flutter,wordpress

# Deep crawl
designlang <url> --depth 5

# Audit my own tokens
designlang lint ./tokens.json
designlang drift <url> --tokens ./tokens.json

# Compare
designlang visual-diff <url-a> <url-b>
designlang brands a.com b.com c.com

# MCP
designlang mcp --output-dir ./design-extract-output
```

Next steps for you:

1. Run `designlang https://vercel.com --emit-agent-rules` in `~/Desktop/` — look at the outputs.
2. Open the `*-preview.html` file — eyeball what was extracted.
3. Try `/extract-design https://linear.app` in Claude Code and watch it run automatically.
4. When you build your next site, run it against a competitor you admire and paste the `.md` file into your Claude Code session as seed context.

Ship something nice.